

PBP Gasal 2026/2027



📄 Proyek Individu > Individual Assignment 2: Implementasi MVT pada Django

Individual Assignment 2: Implementasi Model-View-Template (MVT) pada Django

Pemrograman Berbasis Platform (CSGE602022) - diselenggarakan oleh Fakultas Ilmu Komputer Universitas Indonesia, Semester Gasal 2026/2027

Kontributor: FEIN - Alvin Christian Halim, REM - Malik Alifan Kareem

Tutorial 02 adalah Prasyarat Wajib

Individual Assignment 2 **hanya dinilai apabila Tutorial 02 sudah diselesaikan** paling lambat **Rabu, 9 September 2026**. Kalau Tutorial 02 belum dikerjakan, Individual Assignment 2 **tidak akan dinilai**, meskipun sudah dikumpulkan lewat SCELE.

Turunkan Versi Django Kalau Deployment Gagal

Kalau *deployment* ke PWS gagal karena masalah versi Django, turunkan versinya dengan mengubah baris `django` di `requirements.txt` menjadi:

```
django~=5.0
```

Lalu *push* ulang ke PWS.

Tentang Proyek

Proyek Individu mata kuliah ini adalah sebuah **website portofolio pribadi** yang dikerjakan sendiri. Individual Assignment dirilis tiap minggu mengikuti topik Tutorial minggu itu, dan setiap assignment melanjutkan langsung dari kode yang kamu buat di Tutorial pada minggu yang sama.

Target Tugas Ini

Pada [Tutorial 02](#), kamu sudah menerapkan pola Model-View-Template (MVT) untuk menampilkan data pengalaman (*experience*) pada portofoliomu. Pada tugas ini, terapkan kembali alur yang sama untuk **satu bagian lain** dari portofoliomu, misalnya daftar proyek, pendidikan, sertifikasi, atau bagian lain yang relevan.

Bagian baru tersebut harus memiliki halaman sendiri, terpisah dari halaman utama yang memuat bagian *experience*, dan dapat dibuka melalui *navbar*.

Catatan

Tugas ini hanya mewajibkan **halaman daftar** untuk bagian yang kamu pilih. Halaman detail bersifat opsional dan dapat menjadi fitur tambahan.

Checklist minimal untuk tugas ini:

- ☐ Tambahkan satu model baru pada aplikasi `main` yang merepresentasikan bagian portofolio pilihanmu.
- ☐ Model tersebut memiliki minimal tiga *field* selain *primary key* (`id`, baik dibuat otomatis oleh Django maupun ditentukan sendiri), dengan tipe data yang sesuai.
- ☐ Buat dan terapkan migrasi model, lalu sertakan berkas migrasinya dalam commit.
- ☐ Buat sebuah *view* yang mengambil data dari model, memasukkannya ke dalam *context*, dan meneruskannya ke *template* baru.
- ☐ Tampilkan seluruh objek menggunakan perulangan Django Template Language dan sediakan tampilan untuk kondisi ketika data masih kosong.
- ☐ Data pada bagian portofolio baru tidak ditulis langsung (*hard-coded*) di HTML. Teks antarmuka statis, seperti judul halaman, label navigasi, dan isi *footer*, tetap boleh ditulis di *template*.
- ☐ Daftarkan *named route* pada `main/urls.py` dengan URL yang berbeda dari halaman utama.
- ☐ Tambahkan tautan menuju halaman baru pada *navbar* menggunakan tag `{% url %}`. Pastikan *navbar* dan *footer* konsisten dengan halaman lain.
- ☐ Tambahkan *unit test* yang mencakup minimal tiga kasus pengujian:
 1. URL dapat diakses dan menggunakan *template* yang tepat.
 2. Data model muncul di halaman HTML ketika ada data.
 3. Halaman HTML menampilkan pesan kondisi kosong ketika belum ada data.
- ☐ Pastikan proyek dapat dijalankan dengan `python manage.py runserver` tanpa *error* dan seluruh test lulus ketika menjalankan `python manage.py test`.

Pertanyaan Reflektif

1. Jelaskan alur yang terjadi ketika pengguna membuka halaman portofolio baru, mulai dari permintaan yang diterima proyek hingga data ditampilkan pada *browser*. Dalam jawabanmu, jelaskan peran `urls.py` proyek, `urls.py` aplikasi, *view*, model, dan *template*.
2. Mengapa data untuk bagian portofolio baru sebaiknya disimpan pada model dan tidak ditulis langsung di dalam *template*? Jelaskan dampaknya terhadap kemudahan pemeliharaan dan pengembangan aplikasi.
3. Apa perbedaan fungsi `makemigrations` dan `migrate` pada Django? Berikan contoh perubahan model yang mengharuskanmu menjalankan kedua perintah tersebut.

Jawab pertanyaan-pertanyaan di atas di `README.md` proyekmu (bukan di halaman tugas ini), dengan format berikut supaya tetap rapi seiring bertambahnya pertanyaan reflektif tiap minggu sepanjang semester:

Tugas 2

1. ...
2. ...
3. ...

Pengumpulan

Tugas dikumpulkan lewat slot submisi yang disediakan di SCELE dalam bentuk **satu tautan ke commit** di GitHub, bukan sekadar tautan repositori. Kumpulkan tautan ke **commit paling akhir yang sudah di-push sebelum tenggat waktu** dan menunjukkan hasil akhir tugas ini. Commit tersebut menjadi batas riwayat yang akan diperiksa oleh asisten dosen. Commit yang di-push setelah tenggat waktu tidak akan diterima. Pastikan repositori GitHub kamu bersifat **publik** supaya asisten dosen dapat mengaksesnya untuk penilaian.

Tenggat Waktu Pengerjaan

Individual Assignment 2 dirilis pada **7 September 2026** dan dikumpulkan paling lambat **14 September 2026, pukul 23.59 WIB**. Tenggat waktu ini lebih lama daripada tenggat Tutorial 02, yaitu Rabu, 9 September 2026.

Rubrik Penilaian

Fungsionalitas & Kesesuaian Topik (70%)

- **1** - Tugas belum lengkap; aplikasi gagal berjalan (*crash/error* 500); sama sekali gagal mengimplementasikan materi minggu ini.
- **2** - Aplikasi berjalan sebagian; banyak fitur *error*; implementasi materi minggu ini minim atau salah.
- **3** - Aplikasi berjalan baik dan berhasil mengimplementasikan materi minggu ini, dengan sedikit bug atau fungsi yang kurang mulus.
- **3.5** - Fungsionalitas berjalan sempurna tanpa bug. Semua instruksi dan topik minggu ini diimplementasikan dengan akurat.
- **4** - Melampaui ekspektasi instruksi minggu ini secara kreatif - bebas berkreasi sesuai idemu sendiri, bukan sekadar mengikuti arahan yang tersedia.

Kualitas & Struktur Kode (10%)

- **1** - Kode sangat berantakan; logika tidak jelas; terlihat seperti *copy-paste* tanpa memahami struktur yang mendasarinya.
- **2** - Kode terbaca tapi mengabaikan konvensi *framework* (mis. penamaan variabel acak, kode *spaghetti*).
- **3** - Kode cukup rapi dan terstruktur, mengikuti alur Model-View-Template serta *routing* yang diajarkan di Tutorial 02.
- **3.5** - Struktur kode sangat rapi dan modular; tanggung jawab model, *view*, *template*, dan konfigurasi URL dipisahkan dengan jelas; serta test mudah dibaca.
- **4** - Kualitas kode melampaui ekspektasi minggu ini, mencerminkan kreativitas dan penguasaan yang matang - bukan sekadar mengikuti pola yang diajarkan.

Git & Disiplin (10%)

- **1** - Dikumpulkan terlambat; Git sama sekali tidak dipakai (hanya unggah ZIP/berkas manual).
- **2** - Dikumpulkan tepat waktu; penggunaan Git sangat buruk (mis. cuma 1 *commit* besar dengan pesan tidak jelas).
- **3** - Dikumpulkan tepat waktu; ada beberapa *commit*, tapi pesan *commit* tidak deskriptif (mis. "update", "fix").
- **3.5** - Dikumpulkan tepat waktu. Riwayat *commit* rutin, mencerminkan progres bertahap, dan pesannya sangat deskriptif.
- **4** - Disiplin dan riwayat Git melampaui ekspektasi minggu ini, mencerminkan kematangan dan inisiatif dalam mengelola proyek.

Dokumentasi & AI Disclosure + Pertanyaan Reflektif (10%)

- **1** - **README.md** tidak disediakan; sama sekali tidak menyebutkan penggunaan AI.
- **2** - **README.md** ada tapi tidak ada deskripsi proyek atau instruksi *setup* yang jelas. AI *disclosure* sangat samar (mis. "saya pakai ChatGPT").
- **3** - **README.md** terstruktur dengan deskripsi proyek dan *setup* yang jelas. AI *disclosure* menyebutkan *tools* yang dipakai dan secara umum menyebutkan bagian mana yang dibantu AI.
- **3.5** - **README.md** rapi dan terstruktur dengan instruksi *setup* mingguan yang jelas. AI *disclosure* transparan menjelaskan *tools*, strategi *prompting*, dan bagian spesifik yang dibantu. Menyertakan *chat* AI atau log *prompting*.
- **4** - Dokumentasi dan AI *disclosure* melampaui ekspektasi minggu ini, mencerminkan kedalaman dan inisiatif yang matang.